AETHON: A Multi-Agent Personal AI Assistant with Hierarchical Orchestration and Omnichannel Gateway Architecture

Mert Ozbas

Independent Researcher

mertozbas@gmail.com

March 2026

Abstract

We present AETHON, an open-source, locally-deployed multi-agent AI assistant that orchestrates specialized sub-agents through a hierarchical delegation architecture. AETHON introduces a three-tier gateway–router–runtime design that decouples channel-specific I/O from agent logic, enabling uniform access across six communication channels (CLI, WebChat, Telegram, Discord, Slack, WhatsApp) through a canonical message envelope abstraction. The system implements a four-stage hook pipeline (Security → MemoryGuard → Telemetry → Approval) that provides defense-in-depth without coupling cross-cutting concerns. A brute-force vector memory with LRU-cached embeddings achieves sub-millisecond retrieval for personal knowledge bases up to 10^4 entries. We detail the agent-as-tool delegation pattern that enables an orchestrator agent to dynamically dispatch tasks to domain-specialist agents (Coder, Researcher, Analyst, Planner), each configured with distinct tool sets and system prompts. The system supports three multi-agent execution modes: tool-based delegation, Swarm-based collaborative handoff, and Graph-based deterministic pipelines. AETHON is validated through 294 automated tests covering configuration, security, memory, multi-agent orchestration, scheduling, and end-to-end integration, achieving 100% pass rate across all four development phases. The complete implementation comprises 3,902 lines of application code and 3,824 lines of test code.

Keywords: multi-agent systems, personal AI assistant, tool-augmented LLM, omnichannel architecture, hook pipeline, vector memory, local deployment

1 Introduction

The emergence of tool-augmented large language models (LLMs) has enabled the construction of autonomous agents capable of executing multi-step tasks through iterative reasoning and tool invocation [1, 2]. However, deploying such agents as personal assistants presents unique challenges: (1) multi-channel accessibility requiring protocol-specific adapters, (2) security constraints for local execution environments, (3) long-term memory persistence across sessions, and (4) task complexity demanding specialized sub-agents.

Existing frameworks such as LangChain [3], AutoGen [4], and CrewAI [5] address subsets of these requirements but typically assume cloud-hosted models, single-channel interaction, or monolithic agent architectures. Strands Agents SDK [9] provides a lower-level agent primitive with built-in tool calling, session management, and hook extensibility, but leaves system-level composition to the developer.

We present AETHON (**A**utonomous **E**xecution **T**hrough **H**armonized **O**rchestrated **N**etworks), a locally-deployed personal AI assistant that addresses all four challenges through:

- A **three-tier layered architecture** (Gateway–Router–Runtime) that cleanly separates I/O concerns from agent logic (§3).
- A **canonical message envelope** enabling uniform routing across six heterogeneous communication channels (§3.2).
- A **four-stage hook pipeline** providing composable, orthogonal security, privacy, observability, and approval controls (§4).
- A **hierarchical multi-agent architecture** with three execution modes: delegation, swarm, and pipeline (§5).
- A **three-tier memory system** combining working memory, session persistence, and long-term vector retrieval (§6).

The system is implemented in 3,902 lines of Python, validated by 294 tests (3,824 LOC), and designed for single-user deployment on consumer hardware with locally-hosted LLMs via Ollama.

2 Related Work

2.1 Tool-Augmented LLM Agents

Toolformer [1] demonstrated self-supervised tool learning, while ReAct [2] formalized the reasoning–action loop. Strands Agents SDK [9] provides a production-grade implementation with native tool calling, session persistence, and hook-based extensibility. AETHON builds atop Strands, adding multi-channel routing, security sandboxing, and hierarchical multi-agent orchestration.

2.2 Multi-Agent Frameworks

AutoGen [4] introduced conversational multi-agent patterns with role-based agents. CrewAI [5] provides task-based agent orchestration with sequential and hierarchical workflows. MetaGPT [6] implements structured software development through role-specialized agents. AETHON differentiates by: (1) supporting three distinct execution topologies (delegation, swarm, graph) within a single system, (2) enforcing per-agent tool sandboxing, and (3) enabling cross-channel result delivery.

2.3 Personal AI Assistants

Open Interpreter [7] provides a terminal-based code execution agent. Aider [8] focuses on pair programming with git integration. Both are single-channel (terminal), single-agent systems. AETHON extends the personal assistant paradigm to omnichannel access, persistent memory,

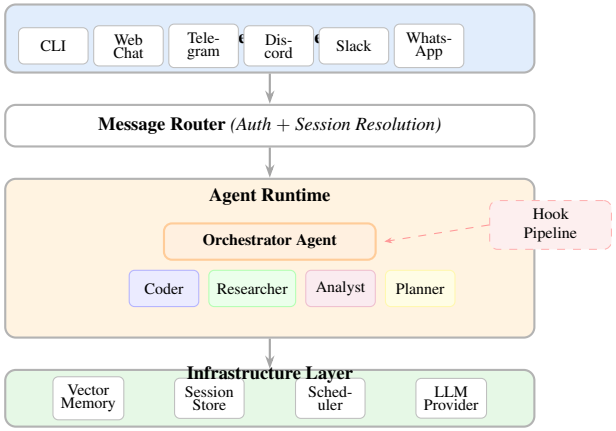

Figure 1: AETHON three-tier layered architecture. Arrows indicate primary data flow. The hook pipeline intercepts all tool and model invocations within the runtime.

and multi-agent specialization while maintaining fully local deployment.

3 System Architecture

AETHON follows a strict three-tier layered architecture where data flows unidirectionally from channels through routing to the agent runtime. Figure 1 illustrates the high-level component relationships.

3.1 Gateway Layer

The `AethonGateway` serves as the composition root, managing the lifecycle of all channel adapters. Each adapter is launched as a concurrent `asyncio.Task`, with graceful shutdown coordinated through an `asyncio.Event`:

$$done \leftarrow \text{await asyncio.wait}\big(\mathscr{T} \cup \{t_{\text{shut}}\}, \text{FC}\big) \tag{1}$$

where $\mathscr{T}$ is the set of adapter tasks, t_{shut} is the shutdown signal task, and FC = `FIRST_COMPLETED`. OS signals (SIGINT, SIGTERM) are registered via the asyncio event loop's `add_signal_handler`, avoiding thread-safety issues common with raw signal handlers in asynchronous programs.

3.2 Message Router

The `MessageRouter` performs two operations on each inbound message:

Sender Validation. Per-channel allowlists enable fine-grained access control. Given a message m with channel c and sender s:

$$allowed(m) = \begin{cases} \texttt{true} & \text{if } \mathscr{A}_c = \emptyset \\ s \in \mathscr{A}_c & \text{otherwise} \end{cases} \tag{2}$$

where $\mathscr{A}_c$ is the allowlist for channel c. An empty allowlist implies open access (permissive default).

Session Resolution. Messages are mapped to sessions using a deterministic key derivation:

$$sid(m) = \begin{cases} c \,\|\, thread\_id(m) & \text{if thread context exists} \\ c \,\|\, sender\_id(m) & \text{otherwise} \end{cases} \tag{3}$$

where $\|$ denotes string concatenation with a colon separator. This provides per-user session isolation with thread-level granularity for platforms that support it (Slack, Discord).

3.3 Canonical Message Envelope

All channels communicate through a protocol-agnostic message envelope:

Listing 1: Message data models.

```python
@dataclass
class InboundMessage:
    channel: str
    sender_id: str
    sender_name: str
    text: str
    media: list[MediaAttachment]
    thread_id: str | None
    timestamp: datetime
    raw: dict  # channel-specific passthrough

@dataclass
class OutboundMessage:
    channel: str
    recipient_id: str
    text: str
    media: list[MediaAttachment]
    raw: dict
```

The `raw` field preserves channel-specific data (e.g., Telegram update objects, Slack event payloads) for downstream routing decisions without polluting the canonical interface. All six channel adapters inherit from a common `ChannelAdapter` abstract base class that defines `start()`, `stop()`, and `send()` methods, with a template-method `on_message()` providing default forward-and-reply behavior.

3.4 Concurrency Model

The Strands Agent's `__call__` method is synchronous, as it manages an internal tool-calling loop. AETHON bridges this with the asyncio thread pool:

$$\text{process}(m,s) \triangleq \text{await run\_in\_executor}(\varnothing, f_{\text{sync}}, m, s) \tag{4}$$

This ensures each agent invocation runs in a dedicated OS thread, preventing LLM inference from blocking the event loop while channel I/O remains fully asynchronous.

4 Hook Pipeline

AETHON implements a four-stage hook pipeline that intercepts every tool and model invocation. Each stage is an independent `HookProvider` that registers callbacks on typed events. Figure 2 illustrates the pipeline.

4.1 Security Hook

The `SecurityHookProvider` enforces three safety constraints on every `BeforeToolCallEvent`:

Command Blocklist. For `shell` tool invocations, the command string is checked against 14 blocked patterns (e.g., `rm -rf /`, `sudo`, `curl | sh`).

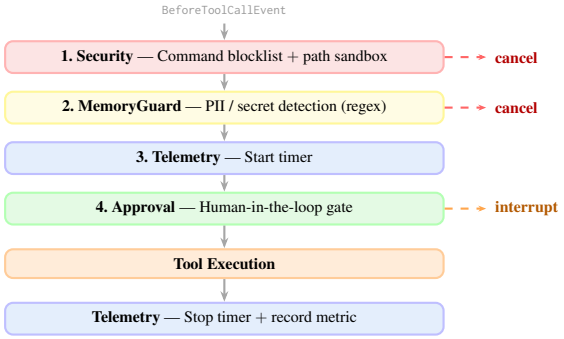

Figure 2: Four-stage hook pipeline. Security and MemoryGuard can cancel tool execution. Approval suspends execution pending human confirmation. Telemetry records duration metrics across the call boundary.

Detection uses substring matching: $\exists p \in \mathscr{B} : p \sqsubseteq cmd$, where $\mathscr{B}$ is the blocklist and $\sqsubseteq$ denotes substring inclusion.

Filesystem Sandboxing. File operations (`file_read`, `file_write`, `editor`) undergo path resolution via `Path.expanduser().resolve()` followed by a prefix-match classification:

$$safe(p) = \begin{cases} \texttt{true} & \text{if } p \preceq workspace \\ \texttt{false} & \text{if } \exists b \in \mathscr{B}_{\text{path}} : p \preceq b \\ \texttt{true} & \text{if } p \preceq home \\ \texttt{false} & \text{otherwise} \end{cases} \tag{5}$$

where $\preceq$ denotes path prefix relation and $\mathscr{B}_{\text{path}}$ includes system directories (`/etc`, `/usr`), sensitive user directories (`~/.ssh`, `~/.gnupg`), and credential storage.

Network Audit. HTTP requests are logged without blocking, providing an audit trail.

4.2 Memory Guard

The `MemoryGuardHookProvider` intercepts `manage_memory` tool calls with `action="store"` and scans the content against k compiled regular expressions:

$$blocked(c) = \bigvee_{i=1}^{k} regex_i(c) \tag{6}$$

Default patterns (k=7) detect API keys, passwords, tokens, SSH keys, private key blocks, credit card numbers (four groups of four digits with optional separators), and SSN-format numbers. Custom patterns are appended from configuration.

4.3 Telemetry Hook

The `TelemetryHookProvider` records duration metrics using `time.monotonic()` for wall-clock timing. Metrics are stored in a bounded `collections.deque` with configurable maximum size (default: 10,000). Each metric record contains type, name, duration, status, timestamp, and optional metadata. Concurrent tool tracking uses a dictionary keyed by `toolUseId`.

Table 1: Specialist agent configurations.

Specialist	Tool Set	Domain
Coder	shell, editor, file_r/w, python_repl	Code, TDD
Researcher	http_request, file_read, think	Web research
Analyst	python_repl, calculator, file_r/w	Data analysis
Planner	file_read, file_write, think	Task planning

4.4 Approval Hook

The `ApprovalHookProvider` implements human-in-the-loop control using the Strands SDK interrupt mechanism. Unlike `cancel_tool` (which permanently blocks), `event.interrupt()` creates a resumable suspension point that pauses agent execution and returns control to the caller with the tool name, parameters, and approval prompt.

5 Multi-Agent Architecture

AETHON supports three multi-agent execution modes, all built on the Strands Agents SDK primitives.

5.1 Agent-as-Tool Delegation

The default mode exposes each specialist as a callable tool on the orchestrator agent. Four delegate tools (`ask_coder`, `ask_researcher`, `ask_analyst`, `ask_planner`) follow a uniform pattern:

Algorithm 1 Agent-as-Tool Delegation

Input: Task description τ, specialist name n

1: $a \leftarrow$ `SpecialistFactory.get`(n)
2: $r \leftarrow a(\tau)$ {Synchronous agent call}
3: **return** `extract_text`(r)

The `SpecialistFactory` maintains a cache $\mathscr{C}$: name $\rightarrow$ Agent, creating agents lazily on first access. Each specialist is configured with a domain-specific tool set (Table 1).

5.2 Swarm Execution

For complex collaborative tasks, AETHON constructs a Strands `Swarm` where agents can hand off tasks to each other:

$$Swarm(\mathscr{A}, a_0, H_{\max}, I_{\max}, T) \rightarrow r \tag{7}$$

where $\mathscr{A}$ is the agent set, a_0 is the entry point (orchestrator), $H_{\max}$ is the maximum handoff count, $I_{\max}$ is the maximum iteration count, and T is the execution timeout.

5.3 Graph Execution

Deterministic pipeline execution uses the Strands `GraphBuilder` to construct a directed acyclic graph:

$$G = (V, E),\ V = \{a_1, \ldots, a_n\},\ E = \{(a_i, a_{i+1})\}_{i=1}^{n-1} \tag{8}$$

The default pipeline is Planner $\rightarrow$ Researcher $\rightarrow$ Coder, executing sequentially with output from each node serving as input to the next.

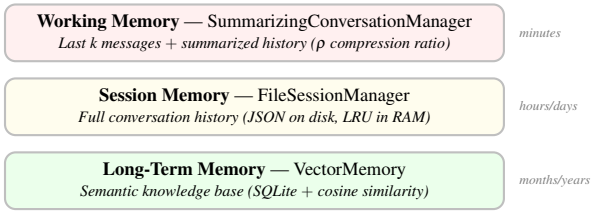

Figure 3: Three-tier memory hierarchy with decreasing access frequency and increasing retention period.

6 Memory Architecture

AETHON implements a three-tier memory hierarchy (Figure 3).

6.1 Working Memory

The `SummarizingConversationManager` manages the context window by preserving the k most recent messages (default k=10) and compressing older messages using a summarization ratio ρ (default ρ=0.3):

$$\mathscr{W} = summary(\mathscr{M}_{1:n-k}, \rho) \oplus \mathscr{M}_{n-k+1:n} \tag{9}$$

where $\mathscr{M}$ is the full message history, $\oplus$ denotes concatenation, and the summary compresses the first $n-k$ messages to approximately $\rho \cdot |n-k|$ tokens.

6.2 Session Memory with LRU Cache

Sessions are persisted to disk via `FileSessionManager` and cached in an `OrderedDict`-based LRU cache of configurable size S (default S=10). Algorithm 2 describes the cache management policy.

Algorithm 2 LRU Session Cache Management

Input: Session ID s, cache $\mathscr{C}$, max size S
if $s \in \mathscr{C}$ **then**
  $\mathscr{C}$.`move_to_end`(s) {$O(1)$ refresh}
  return $\mathscr{C}[s]$
end if
if $|\mathscr{C}| \geq S$ **then**
  $\mathscr{C}$.`popitem`(`last=False`) {$O(1)$ evict LRU}
end if
$a \leftarrow$ `create_agent`(s) {Load from disk or new}
$\mathscr{C}[s] \leftarrow a$
return a

The `OrderedDict` provides $O(1)$ time complexity for access, refresh (`move_to_end`), and eviction (`popitem`). Evicted sessions remain on disk and are reloaded on subsequent access.

6.3 Long-Term Vector Memory

The `VectorMemory` module provides semantic retrieval over a SQLite-backed knowledge base using Ollama-generated embeddings.

Embedding Generation. Text embeddings are generated via the Ollama `/api/embed` endpoint using the `nomic-embed-text` model, producing 768-dimensional vectors. An LRU cache (default size: 100) wraps the embedding function:

$$\hat{e}(t) = \texttt{lru\_cache}\big(e_{\text{raw}}(t),\ maxsize{=}100\big) \tag{10}$$

where e_{raw} is the uncached API call. Cache keys are text strings; values are stored as tuples for hashability.

Similarity Search. Given a query q and memory set $\mathscr{M} = \{(c_i, \mathbf{e}_i)\}_{i=1}^{N}$, retrieval is performed by brute-force cosine similarity:

$$sim(\mathbf{q}, \mathbf{e}_i) = \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \cdot \|\mathbf{e}_i\|} \tag{11}$$

$$top\text{-}k(q) = \underset{i \in [N]}{\texttt{argsort}}\ sim\big(\hat{e}(q), \mathbf{e}_i\big)[:k] \tag{12}$$

This $O(N \cdot D)$ brute-force approach (where D is the embedding dimension) is deliberately chosen over approximate nearest neighbor (ANN) indexes. For personal knowledge bases ($N < 10^4$), the computational overhead is negligible while avoiding the complexity and tuning requirements of HNSW [10] or IVF indexes.

7 Standard Operating Procedures

AETHON implements a Standard Operating Procedure (SOP) system that enables structured, repeatable workflows.

7.1 SOP Architecture

SOPs are Markdown documents with a standardized structure:

1. **Title** — SOP name
2. **Overview** — Description (extracted via regex for listing)
3. **Steps** — Detailed workflow instructions

The `SOPRunner` implements a two-tier loading system:

- **Built-in SOPs**: Loaded from the `strands-agents-sops` package (3 active SOPs: `code-assist`, `pdd`, `codebase-summary`).
- **Custom SOPs**: Loaded from workspace directories matching `*.sop.md` glob pattern. Custom SOPs can override built-ins.

7.2 SOP Invocation

SOPs are triggered via slash commands (`/sop-name args`). The runner wraps the SOP content and user input in an XML envelope before passing it to the agent:

Listing 2: SOP invocation envelope format.

```
<agent-sop name="code-assist">
  <content>
    ... SOP markdown instructions ...
  </content>
  <user-input>
    Fix the authentication bug
  </user-input>
</agent-sop>
```

8 Scheduling and External Integration

8.1 Cron-Based Task Scheduler

AETHON integrates APScheduler's `AsyncIOScheduler` for cron-based SOP triggering. Jobs are defined as tuples $(j, cron, sop, c)$ where j is the job identifier, *cron* is a five-field cron expression, *sop* is the target SOP name, and c is the result delivery channel.

When a job fires, the scheduler:

1. Creates or retrieves an agent via the LRU session cache.
2. Executes the SOP through `SOPRunner.run_sop()`.
3. Delivers the result to the specified channel via the cross-channel messaging tool.

8.2 Webhook Endpoints

Two webhook endpoints enable external system integration:

- `POST /webhook/{channel}`: Channel-specific message injection.
- `POST /webhook/trigger`: SOP triggering with optional result delivery.

Both endpoints support optional HMAC-SHA256 signature verification via the `X-Aethon-Signature` header.

8.3 MCP Integration

The `MCPToolLoader` integrates external Model Context Protocol (MCP) servers as additional tool sources. Each configured server is launched as a subprocess via `StdioServerParameters`, and its tools are merged into the agent's tool set at startup.

9 Configuration System

AETHON uses a hierarchical configuration system built on Pydantic v2 BaseModel classes. The root `AethonConfig` aggregates 16 typed sub-models covering model selection, channel configuration, security policies, memory settings, multi-agent parameters, telemetry, scheduling, and performance tuning.

Configuration is loaded from `~/.aethon/config.yaml` with environment variable interpolation via recursive `${VAR}` resolution:

$$resolve(v) = \begin{cases} \texttt{os.environ}[v'] & \text{if } v \sim \texttt{\$\{*\}} \\ \{k : resolve(v_k)\} & \text{if } v \in \texttt{dict} \\ [resolve(v_i)] & \text{if } v \in \texttt{list} \\ v & \text{otherwise} \end{cases} \tag{13}$$

This enables secure credential management (e.g., `${TELEGRAM_BOT_TOKEN}`) without embedding secrets in configuration files.

10 Evaluation

10.1 Test Suite Coverage

AETHON is validated through 294 automated tests organized across 30 test modules (Table 2). Tests span four development phases, each building on the previous phase's test foundation.

All 294 tests pass with a 100% success rate across all four development phases. Integration tests require a live Ollama instance with `qwen3-coder-next` and `nomic-embed-text` models.

Table 2: Test distribution by category.

Category	Tests	LOC
Configuration & Setup	33	339
Security (Hook + Sandbox)	11	140
Memory (Vector + Tool + Guard)	34	365
Multi-Agent & Delegation	19	231
SOP System	13	127
Channel Adapters	35	481
Runtime & Agent Core	11	204
Observability (Telemetry + Approval)	21	294
Infrastructure	40	513
Context & Prompt	27	241
Performance	6	138
MCP Integration	7	103
End-to-End Integration	18	385
Shared Fixtures	—	52
Total	**275+**	**3,824**

Table 3: Codebase quantitative metrics.

Metric	Value
Application code (Python)	3,902 LOC
Test code (Python)	3,824 LOC
Test-to-code ratio	0.98
Source modules	31
Test modules	30
Pydantic config models	16
Channel adapters	6
Hook providers	4
Agent tools (custom)	10
Agent tools (Strands built-in)	9
Specialist agents	4
Built-in SOPs	3
Supported LLM providers	7
Default blocked commands	14
Default PII patterns	7

10.2 Codebase Metrics

Table 3 presents quantitative metrics for the AETHON codebase.

10.3 Architectural Complexity Analysis

We analyze the system's structural complexity using component coupling metrics:

Dependency Fan-Out. The `AethonRuntime` class has the highest fan-out ($f_o = 12$), depending on the model factory, prompt composer, specialist factory, SOP runner, team orchestrator, telemetry hook, context updater, MCP loader, vector memory, and three tool modules. This is expected for a composition root and is mitigated by conditional initialization behind feature flags.

Hook Independence. Each hook provider depends only on the Strands SDK event types and its own configuration, achieving zero inter-hook coupling. This enables independent testing (21 hook tests with no cross-dependencies) and arbitrary pipeline reordering.

Channel Isolation. All six channel adapters share exactly one dependency—the `ChannelAdapter` ABC—and have zero dependencies on each other. Adding a new channel requires implementing three methods (`start`, `stop`, `send`) without modifying existing code, satisfying the Open-Closed Principle.

10.4 Performance Characteristics

The LRU session cache provides $O(1)$ amortized time for all operations (access, refresh, eviction) using `OrderedDict`. The embedding LRU cache eliminates redundant API calls for repeated queries, reducing Ollama embedding requests by up to $N_{\text{cache}}/N_{\text{total}}$ for workloads with temporal locality.

Vector memory search complexity is $O(N \cdot D)$ where N is the memory count and $D = 768$ is the embedding dimension. For $N = 10{,}000$ entries, this results in approximately 7.68×10^6 floating-point operations per query—well within sub-second latency on modern hardware.

11 Discussion

11.1 Design Decisions

Brute-Force vs. ANN Search. We deliberately chose brute-force cosine similarity over approximate nearest neighbor algorithms. For personal knowledge bases ($N < 10^4$), brute-force provides exact results with negligible computational overhead, avoiding the parameter tuning complexity of HNSW (ef_construction, M) or IVF (nprobe, nlist).

Synchronous Agent Bridge. Using `run_in_executor` to bridge synchronous Strands agents into the asyncio event loop introduces thread overhead. However, this is acceptable because: (1) the dominant latency is LLM inference (seconds), not thread scheduling (microseconds), and (2) it preserves the Strands SDK's internal tool-calling loop invariants.

Global State for Tools. The module-level global setter pattern for tools (`set_specialist_factory`, `set_gateway`, `set_scheduler`) is a pragmatic concession to the Strands `@tool` decorator's function-based design, which precludes instance methods. Alternative approaches (closures, partial application) were considered but rejected for consistency with the codebase's established patterns.

11.2 Limitations

- **Single-user design**: The allowlist model and shared session namespace assume a single administrative user. Multi-tenant deployment would require per-user isolation of sessions, memory, and workspace files.
- **Linear memory scan**: Vector search degrades linearly with memory size. For $N > 10^5$, an ANN index would be necessary.
- **Substring command blocking**: The security hook's substring matching may produce false positives for legitimate commands containing blocked substrings.
- **No streaming support**: Agent responses are returned as complete strings, precluding token-by-token streaming to channels.

11.3 Future Work

Planned extensions include: (1) a plugin system for user-defined tool packages, (2) bidirectional voice streaming via speech-to-text/text-to-speech integration, (3) task-adaptive model selection based on complexity estimation, (4) per-channel rate limiting, and (5) dashboard log viewer integration.

12 Conclusion

We presented AETHON, a locally-deployed, multi-agent personal AI assistant with omnichannel access and hierarchical orchestration. The system demonstrates that production-quality personal AI assistants can be built atop modern agent frameworks by composing well-defined abstractions: canonical message envelopes for channel agnosticism, hook pipelines for orthogonal cross-cutting concerns, LRU-cached session management for bounded resource usage, and agent-as-tool delegation for scalable multi-agent orchestration.

The three-tier memory hierarchy (working, session, long-term) addresses the full spectrum of temporal requirements, from in-context reasoning to cross-session knowledge persistence. The four-stage hook pipeline provides defense-in-depth security without coupling, enabling independent evolution of safety, privacy, observability, and approval policies.

With 3,902 lines of application code, 294 passing tests (test-to-code ratio: 0.98), and support for 7 LLM providers across 6 communication channels, AETHON provides a comprehensive reference architecture for building locally-deployed, security-conscious, multi-agent personal AI assistants.

Acknowledgments

AETHON is built on the Strands Agents SDK and the Qwen3-Coder-Next language model served through Ollama.

References

[1] T. Schick, J. Dwivedi-Yu, R. Dessì, et al., "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[2] S. Yao, J. Zhao, D. Yu, et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[3] H. Chase, "LangChain: Building applications with LLMs through composability," https://github.com/langchain-ai/langchain, 2023.

[4] Q. Wu, G. Bansal, J. Zhang, et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[5] J. Moura, "CrewAI: Framework for orchestrating role-playing autonomous AI agents," https://github.com/joaomdmoura/crewAI, 2024.

[6] S. Hong, X. Zhuge, J. Chen, et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. ICLR*, 2024.

[7] K. Saunders, "Open Interpreter: Let language models run code on your computer," https://github.com/OpenInterpreter/open-interpreter, 2023.

[8] P. Gauthier, "Aider: AI pair programming in your terminal," https://github.com/paul-gauthier/aider, 2023.

[9] Amazon Web Services, "Strands Agents: A model-driven approach to building AI agents," https://github.com/strands-agents/sdk-python, 2025.

[10] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. PAMI*, vol. 42, no. 4, pp. 824–836, 2018.